

Trabalhando com o AIDLC

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [working-with-aidlc.pt-br.pdf](#)

Este guia ajuda você a aproveitar ao máximo o AI-DLC (AI-Driven Development Life Cycle). Ele explica como interagir de forma eficaz com a IA em cada etapa — desde o primeiro prompt até o código funcionando.

Comece pelas instruções básicas em cada seção. As dicas avançadas vêm de experiências reais em workshops e abordam os padrões que as equipes acharam mais úteis depois de se familiarizarem com os fundamentos.

Índice

1. [Regras Gerais](#)
 2. [Fase Inception](#)
 3. [Fase Construction](#)
 4. [Nunca Vibe Code](#)
-

1. Regras Gerais

Fazendo Perguntas Sem Alterar Arquivos

Um dos hábitos mais importantes a desenvolver desde cedo: **nem toda pergunta deve acionar uma atualização de documento.**

Quando você faz uma pergunta à IA sem proteger sua pergunta, ela pode interpretá-la como uma solicitação de mudança e atualizar imediatamente os documentos de design. Para evitar isso, prefixe perguntas exploratórias com uma instrução clara de sem alterações.

Padrão básico:

```
Não atualize nenhum documento. Me ajude a entender por que [esta decisão] foi tomada.
```

```
Não atualize nenhum documento. Para [nome do componente], é razoável usar [biblioteca ou tecnologia] aqui?
```

```
Não altere nada. Avalie o impacto de [mudança proposta].  
Quero entender as consequências antes de decidirmos.
```

Esses padrões permitem que você pense em voz alta com a IA, avalie opções e questione decisões sem se comprometer com nada. Quando estiver satisfeito com a resposta, faça uma instrução deliberada de atualização se necessário.

Dica: Comece cada mensagem exploratória com "Não atualize nenhum documento." Você sempre pode remover essa restrição quando estiver pronto para agir.

O Fluxo Pergunta → Doc → Aprovação

O AIDLC nunca faz perguntas de esclarecimento diretamente no chat. Ele escreve as perguntas em um arquivo markdown e aguarda você preencher suas respostas lá. Isso mantém um registro duradouro de cada decisão e facilita a contribuição de toda a equipe.

Passo 1 — O AIDLC cria um arquivo de perguntas

A IA cria um arquivo como `aidlc-docs/inception/requirements/requirement-verification-questions.md` e para. Ela não prosseguirá até você responder.

Passo 2 — Você preenche suas respostas

Abra o arquivo e preencha cada tag `[Answer]:`. As perguntas usam formato de múltipla escolha:

```
## Question: Deployment model
Onde este serviço será implantado?

A) AWS Lambda (serverless)

B) AWS ECS Fargate (containerizado)

C) Infraestrutura on-premises existente

X) Outro (descreva após a tag [Answer]: abaixo)

[Answer]: B
```

Algumas coisas que funcionam bem ao responder:

- **Adicione um rótulo junto com a letra.** `C - resumo financeiro e cobertura do serviço da dívida` é mais claro do que apenas `C`.
- **Inclua uma breve justificativa.** `A - design-first; gerar a especificação OpenAPI antes de escrever o código` confirma a intenção e fornece contexto que a IA leva adiante.
- **Combine opções quando quiser dizer ambas.** `B e C - rate limiting tanto no nível do API Gateway quanto no da aplicação (não D)` não é ambíguo.
- **Adicione uma ressalva quando a opção estiver quase certa.** `B - a migração é um projeto separado; porém, inclua uma migração única para as novas estruturas de dados.`
- **Use X livremente.** Se nenhuma das opções servir, X é a escolha certa em vez de forçar uma resposta incorreta.

Passo 3 — Diga à IA que suas respostas estão prontas

Volte ao chat e diga: "Respondemos às suas perguntas de esclarecimento. Por favor, releia o arquivo e prossiga."

Dica: pedir explicitamente à IA que *releia* o arquivo garante que ela carregue suas respostas do disco em vez de depender de uma versão em memória que pode não refletir suas edições mais recentes.

Passo 4 — O AIDLC valida e prossegue

A IA lê suas respostas, sinaliza ambiguidades restantes e prossegue para gerar o próximo artefato.

Dica avançada: Se você tiver documentação que responde a algumas das perguntas da IA, pode instruí-la a resolvê-las sozinha: "Analisar a justificativa de cada

pergunta. Se uma pergunta já foi respondida pela documentação fornecida, responda você mesmo. Só me pergunte se ainda não estiver claro." Isso reduz idas e vindas desnecessárias nos pontos de gate.

Gates de aprovação

Ao final de cada etapa, o AIDLC apresenta uma mensagem de conclusão com duas opções:

- **Solicitar Alterações** — peça modificações antes de seguir em frente
- **Aprovar e Continuar** — aceite o output e avance

Leia o artefato gerado antes de aprovar. Discuta com sua equipe se necessário. Aprove apenas quando estiver satisfeito.

Gerenciamento de Contexto

O contexto é a memória de trabalho da IA para a sessão. O AIDLC depende de ter toda a cadeia de artefatos e instruções no contexto para gerar outputs consistentes downstream. Gerenciá-lo bem é um dos hábitos de maior impacto que você pode desenvolver.

A regra central: limpe o contexto em cada ponto de decisão natural.

O AIDLC é construído em torno de gates — momentos em que a IA para e lhe pergunta algo: um arquivo de perguntas para responder, um documento para aprovar, um plano para revisar. Essas pausas não são apenas pontos de verificação de aprovação. Elas são os momentos certos para iniciar um contexto fresco antes de continuar.

Limpar o contexto em um gate tem baixo risco porque o trabalho atual da IA já foi salvo em arquivos. O próximo contexto começa limpo, carrega os artefatos relevantes do disco e prossegue sem carregar ruído acumulado de todas as etapas anteriores.

Se você deixar o contexto acumular por múltiplos gates, a IA começa a trabalhar a partir de uma versão comprimida ou parcialmente perdida das instruções e artefatos anteriores. A qualidade do output degrada de maneiras sutis e difíceis de diagnosticar.

Na prática:

- Quando a IA pede para você responder um arquivo de perguntas — responda as perguntas, depois **inicie um contexto fresco** e diga à IA para reler o arquivo e

continuar

- Quando a IA apresenta um documento para aprovação — revise-o, depois **inicie um contexto fresco** para solicitar mudanças ou aprovar e prosseguir
- Se sua ferramenta oferecer um prompt de "compact context" no meio do workflow, **sempre recuse** — a compactação não é o mesmo que uma reinicialização limpa e perde mais do que economiza

Como retomar após um reset de contexto:

Opção 1 — Método do arquivo de estado (recomendado):

```
Vá para aidlc-docs/aidlc-state.md, encontre o primeiro item não marcado, depois vá ao arquivo de plano correspondente e retome a partir desse ponto.
```

Opção 2 — Entrega manual:

```
Estou retomando uma conversa anteriormente interrompida. Aqui está o contexto:  
[cole o resumo do último output ou mudança recente]  
Por favor, continue com [próxima ação ou seção X].
```

Dica: Faça commit e push de todas as alterações atuais no repositório sempre que resetar o contexto. Leva segundos e significa que você sempre tem um ponto de recuperação limpo.

```
Por favor, faça commit e push de todas as alterações atuais no repositório.
```

Agrupando Prompts

Nem todos os prompts devem ser enviados separadamente. Uma regra simples da experiência de workshops:

Quando duas mudanças estão fortemente acopladas ao mesmo assunto, inclua ambas em um único prompt. Quando duas mudanças não estão relacionadas, faça uma por vez.

Agrupar demais (combinar mudanças não relacionadas) faz a IA perder o foco e deixar passar detalhes. Agrupar de menos (prompts separados para coisas intimamente

relacionadas) adiciona idas e vindas desnecessárias. Em caso de dúvida, opte por separar.

Carregando Arquivos de Referência Externos

Você pode apontar o AIDLC para qualquer documento existente — um schema, um diagrama de arquitetura, um dicionário de dados, uma especificação de API — e ele incorporará esse conteúdo na etapa atual.

Padrão básico:

```
Por favor, leia [caminho do arquivo ou descrição]. Use-o como base para [o que você quer].
```

```
Temos uma estrutura de tabela de auditoria existente. Por favor, adicione-a aos documentos d  
e inception  
e a referencie para este serviço. Quando prosseguirmos, espere novos requisitos e  
histórias relacionadas a este serviço.
```

Dica avançada: Você pode carregar documentos em qualquer etapa, não apenas no início. Se uma nova restrição surgir durante a Construction — uma política de segurança atualizada, um modelo de dados revisado — carregue-a e peça ao AIDLC para avaliar o impacto antes de prosseguir.

Dica avançada — Padrões corporativos como extensões: Se sua organização tiver diretrizes de segurança, conformidade ou API que devem se aplicar a todos os projetos, adicione-as como um arquivo markdown de direcionamento em `aidlc-rules/extensions/`. O AIDLC as carregará automaticamente em cada fase sem exigir injeção manual.

Obtendo Críticas Independentes

O AIDLC vai defender suas próprias decisões anteriores. Quando você quiser uma avaliação imparcial de um artefato, peça uma crítica em um **contexto fresco** — um em que a IA não tem memória de por que tomou essas decisões.

```
Produza um documento de crítica de [o documento de requisitos / o design de componentes].  
Faça isso em um novo contexto separado de tudo o mais.
```

Isso produz feedback mais útil e objetivo do que pedir uma crítica na mesma sessão em que o artefato foi criado.

Níveis de Profundidade

O AIDLC adapta o quão profundamente executa cada etapa com base na complexidade da sua solicitação. Você pode influenciar isso.

```
Mantenha isso em profundidade mínima — só precisamos da estrutura básica documentada.
```

```
Este é um componente crítico para produção. Por favor, execute com profundidade abrangente.
```

2. Fase Inception

A fase Inception é onde você e a IA se alinham sobre *o que construir e por quê* antes que qualquer trabalho de design ou código comece. Quanto mais contexto você trazer aqui, menos perguntas de esclarecimento e menos retrabalho você encontrará na Construction.

Prepare Seus Inputs Antes de Começar

A coisa mais eficaz que você pode fazer antes de iniciar o AIDLC é preparar dois documentos:

1. **Documento de Visão** — o que construir e por quê
2. **Documento de Ambiente Técnico** — quais ferramentas e restrições se aplicam

Esses documentos reduzem dramaticamente o número de perguntas de esclarecimento que o AIDLC fará e garantem que a IA parta do contexto real da sua equipe em vez de fazer suposições.

Por onde começar:

- [writing-inputs/inputs-quickstart.pt-br.md](#) — resumo rápido para greenfield e brownfield

- [writing-inputs/vision-document-guide.pt-br.md](#) — guia completo de visão com modelos
- [writing-inputs/technical-environment-guide.pt-br.md](#) — guia completo de ambiente técnico com modelos

Projetos brownfield (adicionando a uma codebase existente) precisam de inputs ligeiramente diferentes. O documento de visão precisa de uma descrição do estado atual e de uma lista explícita do que não deve mudar. O documento de ambiente técnico deve descrever a stack existente em vez de uma desejada, e exemplos de código devem vir de arquivos existentes reais. Veja [writing-inputs/inputs-quickstart.pt-br.md](#) para o mínimo brownfield e exemplos práticos.

Input mínimo viável se quiser começar rapidamente:

Para a Visão: um parágrafo descrevendo o que você está construindo e para quem, uma lista de funcionalidades do MVP em escopo, uma lista de funcionalidades explicitamente fora do escopo e quaisquer perguntas abertas — coisas que você já sabe que são incertas. Perguntas abertas alimentam diretamente a Análise de Requisitos como ambiguidades pré-declaradas, então elas são resolvidas cedo em vez de surgirem como surpresas no meio do design.

Para o Ambiente Técnico: linguagem e versão, gerenciador de pacotes, framework web, provedor cloud e modelo de deploy, framework de testes, uma tabela de bibliotecas proibidas (com motivo e alternativa recomendada para cada entrada), requisitos básicos de segurança e pelo menos um exemplo de cada para um endpoint típico, função e teste.

A tabela de bibliotecas proibidas importa mais do que uma lista simples — as colunas de motivo e alternativa dizem ao AI-DLC *por que* uma biblioteca é proibida, o que leva a melhores decisões de substituição. Os padrões de código de exemplo são a adição de maior impacto além do básico: eles dão ao AI-DLC um padrão concreto a seguir durante a geração de código em vez de inventar o seu próprio.

Dica: Cada lacuna que você preenche antecipadamente é uma pergunta de esclarecimento a menos durante a Análise de Requisitos.

Iniciando um Novo Projeto

Uma vez que seus documentos de input estejam prontos:

```
Quero iniciar um novo projeto. Por favor, leia [caminho para o documento de visão] e [caminho para o documento de ambiente técnico], depois inicie o workflow do AIDLC.
```

O AIDLC escaneará o workspace, determinará greenfield vs. brownfield e prosseguirá para a Análise de Requisitos usando seus documentos como fonte primária — perguntando apenas sobre o que eles não cobrem.

Para um projeto brownfield, o AIDLC primeiro executará a Engenharia Reversa, analisando sua codebase existente e produzindo documentação de arquitetura, componente e API. Revise esses artefatos cuidadosamente — eles se tornam a base para tudo que segue.

Respondendo às Perguntas de Requisitos

Veja as dicas de resposta na [Seção 1](#) para orientação completa sobre usar letras, adicionar rótulos, combinar opções e usar X para respostas personalizadas. Alguns pontos adicionais específicos para a Análise de Requisitos:

- **Separe explicitamente a visão completa do MVP.** Se o AIDLC perguntar quais funcionalidades incluir, nomeie-as. Se algo está fora do escopo, diga — não deixe ambíguo.
- **Declare claramente decisões deliberadas de "não".** `D - nenhum cache necessário neste momento` sinaliza intenção. Uma resposta em branco convida a IA a fazer uma escolha especulativa.
- **Descreva abordagens em fases inline.** `X - workflow simples baseado em perfis agora; substituir por motor de workflow externo quando disponível` permite que o AIDLC projete a solução atual com os pontos de extensão corretos.

Dica avançada — Extensões de Segurança: Durante a Análise de Requisitos, o AIDLC perguntará se você deseja aplicar regras de extensão de segurança. Para aplicações de nível produção, escolha Sim. Para protótipos, Não está bem. Essa decisão é registrada e aplicada em toda a Construction, então escolha deliberadamente.

Interações Específicas da Inception

Adiando uma funcionalidade no meio do processo:

```
Vamos colocar a capacidade [nome da funcionalidade] no backlog para o release atual.  
Por favor, remova-a do design de componentes e marque as user stories relacionadas como back  
logged.
```

Colocar no backlog (em vez de deletar) preserva o trabalho para iterações futuras sem que ele influencie o build atual.

Registrando uma estrutura de dados existente:

```
Temos um [nome do schema/estrutura] existente. Por favor, adicione-o aos documentos de incep  
tion  
e o referencie para este serviço. Quando prosseguirmos, espere novos requisitos e  
histórias relacionadas a este serviço.
```

Tornando fontes de dados implícitas explícitas:

```
Para o [nome do serviço], adicione o entendimento de que [nova fonte de dados] também é uma  
fonte de dados para esta funcionalidade, além de [fonte de dados existente]. Depois revise  
os requisitos e as user stories para garantir que isso esteja capturado.
```

Verificando impacto upstream após uma mudança de design:

Após qualquer mudança significativa em um artefato de design, peça ao AIDLC para verificar se os documentos anteriores ainda são consistentes:

```
Agora revise as etapas anteriores – user stories e requisitos – para garantir  
que esta mudança não exija atualizações em nenhum desses documentos.
```

Dica avançada — Regra permanente de retropropagação: Em vez de perguntar após cada mudança, defina isso como uma instrução permanente no início de uma fase: "Toda vez que você atualizar um documento, verifique se a mudança impacta o documento de requisitos e as user stories, e me avise se impactar." Isso cria uma rede de segurança automática sem precisar que você se lembre.

Revisão paralela do design de componentes pela equipe:

Se sua equipe se divide para revisar diferentes componentes simultaneamente:

```
Restrinja suas edições aos arquivos sob controle da sua equipe. Quando todas as equipes terminarem,  
pediremos à IA para revisar todas as mudanças e confirmar que não há conflitos.  
Depois pediremos que revise os impactos nas user stories e requisitos.
```

Quando todos terminarem, acione a verificação de conflitos:

```
Tivemos [N] grupos independentes editando arquivos de design de componentes. Por favor, revise todos os arquivos  
e reporte quaisquer conflitos ou inconsistências. Não edite os arquivos – produza um relatório  
para nossa revisão.
```

Resolva cada conflito explicitamente por número:

```
Para o conflito #[número] ([descrição do conflito]):  
atualize [arquivo alvo] para refletir [sua decisão].
```

```
Para o conflito #[número] ([nome da capacidade]):  
esta capacidade está no backlog. Atualize a documentação para marcá-la claramente como  
backlogged para que a geração de código não tente implementá-la.
```

Arquivando arquivos de design desatualizados:

Se a exploração durante o design produziu arquivos que não são mais necessários:

```
Mova as [descrições dos arquivos] para uma pasta de arquivo – não as delete.  
Depois confirme se elas são necessárias para a geração de código.
```

Dica avançada — Restrições de tamanho de componente: Se você quiser evitar componentes superdimensionados que seriam grandes demais para implementar em um único sprint, defina um limite de story points durante o Application Design: "Na fase de design de componentes, injete a seguinte instrução: nenhum componente deve ter mais de [X] story points no total. Se um componente exceder esse limite, desmembre-o em sub-componentes menores."

Dica avançada — Resets de contexto no meio da fase: Se sua sessão for interrompida, use isso para restabelecer o estado:

```
text Pare. Novo contexto. Acabamos de concluir [descrição do trabalho recente]. Por favor,  
revise [artefatos upstream] para avaliar qualquer impacto da mudança recente. [Cole a  
descrição da mudança aqui.]
```

3. Fase Construction

A fase Construction é onde os designs se tornam código. Cada unidade de trabalho passa por uma série de etapas de design (condicionais) seguidas da Geração de Código (sempre). Após todas as unidades serem concluídas, Build and Test encerra o trabalho.

O Processo de Revisão de Design

Para cada unidade de trabalho, o AIDLC pode executar algumas ou todas estas etapas de design antes de gerar código:

- **Functional Design** — lógica de negócio, modelos de domínio, schemas de dados
- **NFR Requirements** — performance, segurança, escalabilidade, seleção de stack tecnológica
- **NFR Design** — aplicando padrões NFR ao design
- **Infrastructure Design** — mapeando o design para serviços cloud reais

Cada etapa produz um documento em `aidlc-docs/construction/{unit-name}/`. Seu trabalho em cada gate é ler o documento e decidir: solicitar mudanças ou aprovar.

Leia antes de aprovar. Os documentos de design são a fonte de verdade para a geração de código. Erros que passam despercebidos aqui são mais difíceis de corrigir depois.

Avançando do design para o código:

Quando estiver pronto para fazer a transição para a Geração de Código, forneça à IA o contexto estrutural necessário antecipadamente:

```
Concluimos a revisão do design de componentes. Estamos prontos para a criação de código.  
Por favor, use a seguinte estrutura de diretórios e código-fonte:  
[referencie um serviço existente ou estrutura de pastas].  
Use este padrão para APIs. Para a UI, siga a estrutura de diretórios [Vue.js composables/com  
ponents/store].  
Por favor, faça quaisquer perguntas que tiver antes de prosseguir.
```

Convidar perguntas antes do início da geração resolve ambiguidades no plano em vez de no meio da criação de arquivos.

Solicitando uma correção direcionada:

Seja preciso — nomeie o elemento, o que está errado e o que deveria ser:

```
O [descrição do endpoint] deve usar [parâmetro correto], não [parâmetro incorreto].  
Por favor, atualize o [nome do componente] de acordo.
```

Escolhendo entre opções apresentadas pela IA:

```
Por favor, implemente a Opção B — [descrição da opção] — para [nome da funcionalidade].  
Atualize todos os documentos de design de componentes de acordo.
```

Referencie a opção pela letra e pela descrição, e escope explicitamente a atualização para todos os documentos afetados, não apenas aquele onde a pergunta surgiu.

Substituindo um padrão de design:

```
Preferimos nos desviar de [padrão padrão] e usar [nossa abordagem preferida]  
para permitir [justificativa]. Por favor, atualize os documentos de design de componentes de  
acordo.
```

A justificativa importa. O AIDLC a leva adiante para etapas posteriores, o que evita que o desvio seja silenciosamente revertido.

Dica avançada — Avaliação de impacto antes de comprometer: Para qualquer mudança de design significativa, avalie antes de agir:

```
text Não altere nada. Avalie o impacto de [mudança proposta]. [Descreva a mudança proposta  
em detalhes.]
```

Dica avançada — Documentação de código inline: Se você quiser documentação inline aplicada consistentemente a cada unidade, adicione-a como uma regra permanente no início da fase Construction em vez de repeti-la por unidade: "Adicione documentação de código inline como regra padrão para a fase de construction."

O Processo de Geração de Código

A Geração de Código tem duas partes distintas. Ambas requerem sua aprovação explícita.

Parte 1 — Planejamento

O AIDLC cria um plano numerado e rastreado por checkboxes de cada arquivo a ser criado ou modificado. Revise este plano antes de aprovar. Verifique se:

- Cada arquivo está no local correto (código da aplicação na raiz do workspace, nunca em `aidlc-docs/`)
- As etapas cobrem tudo que seus documentos de design especificaram
- Projetos brownfield listam arquivos existentes a modificar, não novos duplicatas ao lado deles

Dica avançada — Bibliotecas internas: Antes de aprovar o plano, injete seus requisitos de biblioteca interna no arquivo de Q&A ou plano de implementação:

```
text Além das minhas respostas, você deve usar as seguintes bibliotecas do nosso [projeto de partida / building blocks]: [liste cada biblioteca explicitamente]. Explique por que e quando cada uma deve ser usada, não apenas o que é.
```

Um guia markdown curado das suas bibliotecas internas funciona melhor do que apontar a IA para um repositório. Crie um e referencie-o como input de geração de código.

Dica avançada — UI a partir de designs Figma: Tire um screenshot do seu design Figma, passe-o para um modelo com capacidade de visão (ex.: ChatGPT) para gerar código de framework a partir do screenshot, depois forneça esse output ao AIDLC como input de implementação de UI. Isso produz uma especificação concreta e legível por ferramentas em vez de uma exportação bruta de ferramenta de design.

Parte 2 — Geração

O AIDLC executa cada etapa sequencialmente, marcando cada etapa ao concluir. Quando todas as etapas estão concluídas, apresenta a mensagem de conclusão com caminhos para os arquivos gerados.

Revise o código gerado antes de aprovar. Se algo não estiver certo:

```
Solicitar Mudanças: [descreva especificamente o que precisa mudar]
```

Dica avançada — Modificações de arquivos brownfield: Para codebases existentes, o AIDLC modifica arquivos no lugar. Se você ver `ClassName_modified.java` ou `service_new.ts` ao lado do original, sinalize imediatamente:

```
text Vejo [ClassName_modified.java] ao lado de [ClassName.java]. Por favor, mescle as  
mudanças no arquivo original e delete a duplicata.
```

Build and Test

Após todas as unidades serem concluídas, o AIDLC gera instruções de build e teste para todas as unidades. Alguns padrões úteis a saber:

Injetando ferramentas de teste no momento certo:

Não adicione instruções de framework de teste ou sistema de gerenciamento de testes no início do projeto. Quando a geração de código começar, esses detalhes podem ter sido comprimidos ou perdidos ao longo de muitas etapas intermediárias. Injete-os no momento certo:

```
Na etapa de geração de testes funcionais, injete a seguinte instrução:  
gere testes funcionais usando o formato [sistema de gerenciamento de testes] descrito  
neste documento: [anexe a especificação]. Use este endpoint de API para enviar  
os casos de teste gerados para o repositório [sistema de gerenciamento de testes]: [detalhes  
do endpoint].
```

Este princípio se aplica a qualquer instrução específica de ferramenta: injete-a na fase onde é necessária, não no início do projeto.

Definindo escopo de cobertura de testes unitários:

```
Ao gerar testes unitários, exclua dependências externas de terceiros dos cálculos de cobertura de código. Exija um mínimo de 80% de cobertura apenas nos caminhos de código internos.
```

Após a Geração de Código: Retropropagando Mudanças

Mudanças feitas durante a geração de código — pequenas decisões de design, ajustes descobertos ao escrever código — precisam fluir de volta para os documentos de design. Faça isso como uma varredura deliberada após o polimento do código estar completo, não ad hoc:

```
Quando terminar de polir o código, revise os arquivos de design final de cada unidade e propague quaisquer mudanças de volta para os requisitos e user stories. Faça um plano de como fazer isso passo a passo antes de executar.
```

Pedir um plano antes da execução garante que a varredura seja sistemática em todas as unidades em vez de seletiva.

Dica avançada — Extraíndo especificações reutilizáveis: Ao final de um projeto concluído, extraia os padrões que você estabeleceu em documentos de especificação reutilizáveis para projetos futuros:

```
text Crie um conjunto de documentos de especificação reutilizáveis a partir dos padrões expressos neste projeto: um para design de API, um para segurança, um para especificações de UI, um para a stack de tecnologia e um para a estrutura de diretórios. Use as unidades concluídas como fonte. Vou revisar e aprovar cada documento antes que seja usado em projetos futuros.
```

4. Nunca Vibe Code

Vibe coding significa editar diretamente os arquivos de código gerado para fazer correções rápidas ou experimentar coisas — contornando completamente os documentos de design. Parece rápido no momento e cria problemas logo depois.

O problema não é a edição em si. É que os documentos de design — a fonte de verdade que o AIDLC usa para cada operação subsequente — não refletem mais o que o código

realmente faz. Na próxima vez que o AIDLC executar a Geração de Código para uma unidade relacionada, ou você retomar uma sessão, ou um colega pegar o trabalho, a desconexão causa confusão e retrabalho.

Uma equipe descreveu diretamente durante os workshops:

"Você nunca corrige o código diretamente. Se descobrir um problema, volte ao AIDLC e diga: Descobri o problema X. Revise o design e faça um plano para corrigi-lo. Se isso afeta o design, atualize-o, depois atualize o código."

A regra: atualize o design primeiro, depois gere o código.

A Forma Correta de Fazer uma Mudança

Seja você descobriu um bug, mudou de ideia sobre uma decisão de design ou recebeu novos requisitos, o fluxo é o mesmo:

Passo 1 — Descreva o problema sem tocar em nada:

```
Não atualize nenhum documento ainda. Descobri o problema [X].  
Revise o design e me ajude a entender onde isso precisa ser tratado.
```

Passo 2 — Corrija o documento de design:

```
Por favor, atualize [documento de design específico] para refletir [a correção].  
Depois verifique se quaisquer documentos upstream – requisitos, user stories –  
também precisam ser atualizados.
```

Passo 3 — Regere o código afetado:

```
O design para [nome da unidade] foi atualizado. Por favor, reexecute a geração de código  
apenas para os arquivos afetados.
```

Esse fluxo leva alguns minutos a mais em comparação com editar diretamente um arquivo. Mantém sua documentação sincronizada, sua trilha de auditoria completa e sua equipe alinhada sobre o que foi realmente construído.

Quando Você Está Tentado a "Simplesmente Editar o Arquivo"

"É apenas uma correção de uma linha."

Correções de uma linha que contornam o design ainda criam divergência. Registre a correção no documento de design relevante e deixe o AIDLC aplicá-la:

```
Em [functional-design.md para a unidade X], atualize [método ou regra] para [a correção].  
Depois regere [o arquivo afetado].
```

"Estamos apenas explorando — nada é final ainda."

Exploração é exatamente para o que "Não atualize nenhum documento" serve. Explore livremente no chat. Comprometa-se apenas quando estiver pronto.

"Preciso desbloquear a equipe agora."

Às vezes você precisa agir rápido. Se fizer uma edição direta, registre-a honestamente para que a trilha de auditoria permaneça precisa:

```
Fizemos uma edição direta temporária em [arquivo] para desbloquear a equipe.  
A correção foi [descrição]. Por favor, atualize [documento de design] para refletir isso  
e verifique se nenhum outro documento está inconsistente.
```

Regras Permanentes que Previnem Divergência

Duas instruções permanentes que você pode definir no início de uma fase Construction que capturam problemas cedo, sem precisar que você se lembre de perguntar cada vez:

Retropropagação em cada atualização:

```
Toda vez que você atualizar um documento, verifique se a mudança impacta o  
documento de requisitos e as user stories, e me avise se impactar.
```

Design primeiro em cada decisão de código:

```
Quando você tomar uma decisão de design durante a geração de código, sempre certifique-se de  
que a documentação reflita essa mudança antes de prosseguir.
```

Defina-as uma vez no início da Construction e elas se aplicam para toda a fase.

Mantendo Relatórios Fora do aidlc-docs

Uma nota prática: se você pedir ao AIDLC para produzir relatórios voltados para humanos — diagramas de arquitetura, resumos de componentes, apresentações para stakeholders — não deixe que ele os salve em `aidlc-docs/`. Esses arquivos serão carregados como artefatos em etapas subsequentes, inflando o número de tokens e potencialmente confundindo a IA sobre o que é input de design autoritativo.

Use uma pasta separada `reports/` e, para um output mais limpo, gere relatórios em um contexto fresco com um arquivo de especificação de relatório dedicado:

```
Pause o processo. Inicie um novo contexto. Leia [arquivo markdown de especificação de relatório]
e produza o relatório com base no estado atual dos artefatos AIDLC.
Salve o output em uma pasta reports/, não em aidlc-docs/.
```

Para guias sobre como preparar seus documentos de input, veja [writing-inputs/inputs-quickstart.pt-br.md](#).